Instrucţiuni de salt

Comportament general

- modifică valoarea registrului contor program
  - deci adresa următoarei instrucţiuni care va fi executată
- categorii
  - salt necondiționat - întotdeauna
  - salt condiționat - doar în anumite situații

Salt necondiţionat

- sintaxă

`jmp` *`adresă`*

- exprimarea adresei - prin etichete
  - similar limbajului C (instrucțiunea `goto`)
  - se pot face salturi din blocuri de cod `_asm` în blocuri de cod C și invers

Salturi condiţionate

- se testează o condiție și se face saltul doar în cazul în care condiția este adevărată
  - sau doar dacă este falsă
- în caz contrar execuția continuă cu instrucțiunea următoare
- ce condiții putem testa?
  - indicatorii de condiții (luați individual)
  - sau combinații ale acestora

Pentru indicatori individuali

indicator testat	salt pt. valoarea 1	salt pt. valoarea 0
Carry	`jc`	`jnc`
Overflow	`jo`	`jno`
Zero	`jz`	`jnz`
Sign	`js`	`jns`

- în toate cazurile trebuie precizată adresa de salt

Condiții compuse

- limbajul C - teste pe baza operatorilor relaţionali

  <, <=, ==, !=, >, >=

- nu este suficientă testarea unui singur indicator
  - dar relațiile pot fi determinate verificând toți indicatorii

Comparare

- sintaxă

`cmp` *`op1,op2`*

- realizează intern o scădere
  - nu scrie rezultatul nicăieri
  - însă setează indicatorii de condiții
- instrucțiunea trebuie urmată de un salt care testează o anumită relație între *`op1`* și *`op2`*

Salturi care testează relații

relaţie	operanzi fără semn	operanzi cu semn
op1 < op2	jb	jl
op1 <= op2	jbe	jle
op1 > op2	ja	jg
op1 >= op2	jae	jge
op1 == op2	je	je
op1 != op2	jne	jne

Structuri de control

- în limbajul C nu se recomandă utilizarea instrucțiunii `goto`
- în schimb, se folosesc structuri de control
- dar implementarea acestora la nivelul procesorului se face tot prin instrucțiuni de salt

Structura If (1)

- implementare - salt condiționat pe condiție inversă celei din limbajul C
- motivație
  - în limbajul C, se execută o secvență de cod dacă o condiție este adevărată
  - în limbajul de asamblare, instrucțiunile de salt pot evita execuția unei secvențe de cod

Exemplu

- cod C

```
int a,x;
// tip cu semn
...
if(x>5)
  a=2;
```

- cod ASM

```
cmp x,5
jle maimare
mov a,2
maimare:
```

Structura If (2)

- dacă avem şi o ramură `else`, trebuie utilizate două instrucţiuni de salt
- a doua instrucţiune se asigură că, la terminarea execuţiei instrucţiunilor din prima ramură, nu vor fi executate şi instrucţiunile din a doua ramură
  - salt necondiționat

Exemplu

- cod C

```
if(x>5)
  x--;
else
  x++;
```

- cod ASM

```
cmp x,5
jle nu
dec x
jmp afara
nu:
inc x
afara:
```

Structura While

- două instrucţiuni de salt
  - prima pentru a decide dacă se execută o nouă iteraţie sau se părăseşte bucla
    - saltul - pe condiţia inversă celei din codul C
  - a doua pentru a relua execuția de la început
    - la finalul corpului buclei
    - salt necondiţionat

Exemplu

- cod C

```
while(x<10)
  x++;
```

- cod ASM

```
bucla:
cmp x,5
jge afara
inc x
jmp bucla
afara:
```

Structura For

- similară structurii While
- instrucţiuni auxiliare - delimitează clar
  - iniţializarea variabilelor
  - actualizarea valorilor variabilelor pentru iteraţia următoare

Exemplu

- cod C

```
for(i=0;i<=5;i++)
  s+=3;
```

- cod ASM

```
mov i,0
bucla: cmp i,5
jg afara
add s,3
inc i
jmp bucla
afara:
```

Structura Do-While

- testul de continuare/ieşire - la final
- se testează aceeaşi condiţie ca în limbajul C
  - în ambele limbaje, condiția adevărată duce la reluarea buclei

Exemplu

- cod C

```
do {
  s+=i;
  i++;
  } while(i<=5);
```

- cod ASM

```
bucla:
mov eax,s
add eax,i
mov s,eax
inc i
cmp i,5
jle bucla
```